战队名称

智能工人

比赛排名

总分数：2083

大一新生组排名：8

总排名：33

战队成员

Shy、曹洪荣

(Trae、
Doubao)

签到

1、下载附件。

2、打开网页。

3、上传照片。

4、复制 flag。

babyRSA

1、把问题输给 trae。

2、新建.js 文件，复制代码。

3、调试，把错误结果复制给 ai，apply，直至跳出 flag。

.js 代码：

```
// RSA 广播攻击实现 - JavaScript 版本

// 当使用相同的 e 值(3)加密相同消息到不同模数时，可以利用中国剩余定理恢复明文



// 输入值

const e = 3;

const N1 =
19831146070473780450608271965513333016477420167810804693209535211751519378798100050738385201161854775668250
18449161299371084493353215563878704521948670778742028205817761183896079032073462480979631092689461550701439
39054391641083053431470562851870480 3n;

const C1 =
13192445407076444144417541514427188094854353253584529650871742429298429176122106888115309616831240701274243
77191377029876445462571124888103255630444630358056166512688667093604360446881202411111644335827267412155439
60711466178539397565378934310918395 5n;

const N2 =
66111404265300574613935728983655950688111510106384565086238514901421964133746188531859108773677252707542147
42342668772293348314346742063891792348563973154553329108954734139316568648053349192410348374700962018349800
07626796062875935772391134760620473 3n;

const C2 =
41998123843875621116729989501531316956049116280732482300490270393719879500976089055100523216197820417935038
24852700662126577132812515825278789093650061665719100684998487448788789790831650358875872761371787673450056
83945570920813989625378505592054310 4n;

const N3 =
14126749535297606509514418674348551918358936748316341471639452521057767873962355396372423101375036984635365
37639777338454133030405662752295698006210425914786710035726827178239512351628469990235459966710606162018056
65493329903786924467821966991161411 7n;

const C3 =
41663026452800366344084116174180504236555705080505184877204086431216138707209002745841467383559886160630317
64515988172971348800237604645253610288435108772191651156927479743851176000265117463877466341851549147172873
34233494293011261623790319302692124n;
```

```
// 扩展欧几里得算法 - 计算模逆元
```

```
function extendedGCD(a, b) {
    if (b === 0n) {
        return [a, 1n, 0n];
    } else {
        const [g, x, y] = extendedGCD(b, a % b);
        return [g, y, x - (a / b) * y];
    }
}
```

```
// 中国剩余定理实现
function crt(remainders, mods) {
    let result = 0n;
    let product = 1n;

    // 计算所有模数的乘积
    for (const mod of mods) {
        product *= mod;
    }

    for (let i = 0; i < mods.length; i++) {
        const mod = mods[i];
        const rem = remainders[i];
        const m = product / mod;

        // 计算 m 在 mod 下的逆元
        const [g, x, y] = extendedGCD(m, mod);
        if (g !== 1n) {
            throw new Error('模数不互质');
        } else {
```

```
            const inv = x % mod;

            result = (result + rem * m * inv) % product;

        }

    }



    return result;

}
```

```
// 计算整数的立方根

function integerCbrt(n) {

    if (n === 0n) return 0n;

    if (n < 0n) return -integerCbrt(-n);



    let low = 1n;

    let high = n;

    let result = 0n;



    while (low <= high) {

        const mid = (low + high) / 2n;

        const midCube = mid * mid * mid;



        if (midCube === n) {

            return mid;

        } else if (midCube < n) {

            low = mid + 1n;

            result = mid;

        } else {

            high = mid - 1n;

        }
```

```
    }

    // 检查 result 和 result+1 哪一个更接近 n
    const resultPlusOneCube = (result + 1n) * (result + 1n) * (result + 1n);
    if (resultPlusOneCube === n) {
        return result + 1n;
    }

    return result;
}
```

```
// 将数字转换为 ASCII 字符串
function numberToString(num) {
    let str = '';
    let n = num;
    while (n > 0) {
        const charCode = n % 256n;
        str = String.fromCharCode(Number(charCode)) + str;
        n = n / 256n;
    }
    return str;
}
```

```
// 主函数
function main() {
    try {
        // 使用中国剩余定理求解同余方程组
        // 找到一个数 x，使得 x ≡ C1 mod N1, x ≡ C2 mod N2, x ≡ C3 mod N3
        const x = crt([C1, C2, C3], [N1, N2, N3]);
```

```
        // 计算 x 的立方根得到明文 m
        const m = integerCbrt(x);

        // 验证结果
        console.log('计算得到的 m:', m);
        console.log('验证: m^3 mod N1 =', (m * m * m) % N1 === C1 ? '正确' : '错误');
        console.log('验证: m^3 mod N2 =', (m * m * m) % N2 === C2 ? '正确' : '错误');
        console.log('验证: m^3 mod N3 =', (m * m * m) % N3 === C3 ? '正确' : '错误');

        // 将明文转换为字符串
        const plaintext = numberToString(m);
        console.log('明文字符串:', plaintext);

        return {
            m: m.toString(),
            plaintext: plaintext
        };
    } catch (error) {
        console.error('错误:', error.message);
    }
}
```

```
// 执行主函数
const result = main();
console.log('最终结果:', result);
```

Remaining thing

1、把问题输给 trae。

2、新建.js 文件，复制代码。

3、调试，把错误结果复制给 ai，apply，直至跳出 flag。

.js 代码：

```
// 输入参数

const N =
636585149594574746909030160182690866222909256464847291783000651837227921337237899651287943597773270944384034
858925295744880727101606841413640006527614873110651410155893776548737823152943797884729130149758279127430 0
447392540004266109228345730949570825895394456108282794288145243134912620619305128290744662326331305991044 90
893572093943832740301809630847541592548921200288222432789208650949937638303429456468889100192613859073752 92
381245421223990894893017835533139093353677106579181764397876304503083371232616288381063812002937833709293 86
621741197476878994846036283440794935566014224984053607319581627192961605840426710571602412848525229136762 64
596201906163n;

const m1 =
437018910555377754758713090674828434552623609418189605883878395265943244846522356956125593648928796885738 90
427519579032240390473157740622166280061564919213509659475856799070077471045862160654480499532431923020217 54
088066667067608779792257328070836327813390658314167598564599220144143214194506505311268334519296403564776 52
908076978264462932182064696092260867124176080216707230318582489201642918604678896800531886883658762692711 48
411366521114579941515095579423120347532691005071455299078906630639750009751068833180938058753804173528065 23
685326674344204327480555626216929911748510640176509890886868972962874987772280199858697989560677638118791 78
829955994366n;

const m2 =
423775156887345771483214073174563566225971899378137973113300330050744806291600501030439300493007373439572 48
657685490037718390056812487564858040511790033476355073470485054552519531228228601891869262676675469871579 76
752426499878301232540750748492601324652727990505357156127909059250768098083685196252399118126351744996553 30
946748878170282554491220873730105594218997553839772828068688676446878747560971142890996647615848402287281 88
409340682601741183888960924986367731785233652449745699607502213095604706002750379157142864532271337781634 43
489032240593311116566239168855132482884514907219444505525063000466060508375157092179645411770435125441473 27
439793371612n;

const e = 0x10001n; // 65537



// 扩展欧几里得算法，用于计算模逆元

function extendedGCD(a, b) {

    if (b === 0n) {

        return [a, 1n, 0n];

    } else {

        const [g, x, y] = extendedGCD(b, a % b);
```

```
        return [g, y, x - (a / b) * y];
    }
}
```

```
// 计算模逆元
function modInverse(a, m) {
    const [g, x, y] = extendedGCD(a, m);
    if (g !== 1n) {
        throw new Error('模逆元不存在');
    } else {
        return x % m;
    }
}
```

```
// 快速幂取模
function powMod(base, exponent, mod) {
    let result = 1n;
    base = base % mod;
    while (exponent > 0n) {
        if (exponent % 2n === 1n) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exponent = exponent / 2n;
    }
    return result;
}
```

```
// BigInt 版本的平方根计算 - 使用二分查找法
```

```
function bigIntSqrt(n) {
    if (n < 0n) {
        throw new Error('负数没有平方根');
    }
    if (n === 0n || n === 1n) {
        return n;
    }

    // 二分查找范围
    let low = 1n;
    let high = n / 2n;
    let result = 0n;

    while (low <= high) {
        const mid = (low + high) / 2n;
        const midSquared = mid * mid;

        if (midSquared === n) {
            return mid;
        } else if (midSquared < n) {
            // 尝试下一个更大的值，但记录当前找到的最接近的结果
            low = mid + 1n;
            result = mid;
        } else {
            high = mid - 1n;
        }
    }

    // 检查 result+1 是否更接近
    const nextResult = result + 1n;
```

```
    if (nextResult * nextResult === n) {
        return nextResult;
    }

    return result;
}
```

```
// 分解 N，尝试 Fermat 分解法（适用于 p 和 q 相差较小的情况）
function fermatFactor(n) {
    // 由于题目中提到 p 和 q 是相邻的素数（nextprime 关系），它们应该很接近
    // 使用 bigIntSqrt 代替 Math.sqrt 来计算初始 a 值
    let a = bigIntSqrt(n);
    if (a * a < n) {
        a += 1n; // 确保 a² >= n
    }

    console.log('开始 Fermat 分解，初始 a 值:', a);

    let attempts = 0n;
    const maxAttempts = 100000n; // 设置最大尝试次数，避免无限循环

    while (attempts < maxAttempts) {
        const b2 = a * a - n;
        const b = bigIntSqrt(b2);

        if (b * b === b2) {
            const p = a + b;
            const q = a - b;
            if (p * q === n) {
```

```
            console.log(`Fermat 分解成功！尝试次数：${attempts}`);
            return [p, q];
        }
    }

    a += 1n;
    attempts += 1n;

    // 每 1000 次尝试显示一次进度
    if (attempts % 1000n === 0n) {
        console.log(`已尝试 ${attempts} 次...`);
    }
}

throw new Error(`Fermat 分解失败，已尝试${maxAttempts}次，p 和 q 可能相差较大`);
}
```

```
// 将长整数转换为字符串
function numberToString(num) {
    let str = '';
    let n = num;
    while (n > 0n) {
        const charCode = n % 256n;
        str = String.fromCharCode(Number(charCode)) + str;
        n = n / 256n;
    }
    return str;
}
```

```
// 主函数

function main() {

    try {

        console.log('开始分解 N...');

        console.log('N 的位数:', N.toString().length);



        // 分解 N 得到 p 和 q

        const [p, q] = fermatFactor(N);

        console.log('分解成功! ');

        console.log('p =', p.toString().substring(0, 50) + '...');

        console.log('q =', q.toString().substring(0, 50) + '...');



        // 验证分解是否正确

        const checkN = p * q;

        console.log('分解验证:', checkN === N ? '正确' : '错误');



        // 计算欧拉函数

        const phi = (p - 1n) * (q - 1n);

        console.log('计算得到 phi(N)...');



        // 计算私钥 d

        const d = modInverse(e, phi);

        console.log('计算得到 d...');



        // 解密 m1 和 m2 得到 c1 和 c2

        console.log('开始解密 c1...');

        const c1 = powMod(m1, d, N);

        console.log('开始解密 c2...');

        const c2 = powMod(m2, d, N);

        console.log('解密得到:');
```

```
console.log('c1 = F1 + F2 =', c1);

console.log('c2 = F1³ + F2³ =', c2);


// 利用数学公式恢复 F1 和 F2

// 已知：F1 + F2 = c1

//       F1³ + F2³ = c2

// 利用公式：F1³ + F2³ = (F1 + F2)³ - 3*F1*F2*(F1 + F2)

// 可以得到：F1*F2 = ((c1³ - c2) / (3*c1))

console.log('计算 c1³...');

const c1Cubed = c1 * c1 * c1;

console.log('计算 F1*F2...');

const product = (c1Cubed - c2) / (3n * c1);

console.log('计算得到 F1*F2 =', product);


// 现在我们有一个二次方程：x² - c1*x + product = 0

// 求解这个方程的两个根，即为 F1 和 F2

// 使用求根公式：x = [c1 ± sqrt(c1² - 4*product)] / 2

console.log('计算判别式...');

const discriminant = c1 * c1 - 4n * product;

console.log('判别式 =', discriminant);


// 计算判别式的平方根 - 使用 bigIntSqrt 函数

console.log('计算判别式的平方根...');

const sqrtDiscriminant = bigIntSqrt(discriminant);

console.log('判别式的平方根 =', sqrtDiscriminant);


// 验证平方根是否正确

if (sqrtDiscriminant * sqrtDiscriminant !== discriminant) {

    throw new Error('平方根计算错误，请检查计算过程');

}
```

```
// 计算两个根
console.log('计算 F1 和 F2...');
const F1 = (c1 + sqrtDiscriminant) / 2n;
const F2 = (c1 - sqrtDiscriminant) / 2n;
console.log('计算得到:');
console.log('F1 =', F1);
console.log('F2 =', F2);

// 验证计算结果
console.log('验证 c1 = F1 + F2:', (F1 + F2) === c1 ? '正确' : '错误');
console.log('验证 c2 = F1³ + F2³:', (F1*F1*F1 + F2*F2*F2) === c2 ? '正确' : '错误');

// 转换为字符串
const flag1 = numberToString(F1);
const flag2 = numberToString(F2);
console.log('恢复的前半部分 flag:', flag1);
console.log('恢复的后半部分 flag:', flag2);

// 拼接完整的 flag
const fullFlag = flag1 + flag2;
console.log('完整的 flag:', fullFlag);

// 验证 flag 长度是否符合题目要求
console.log('flag 长度:', fullFlag.length, '（题目要求 38）');

return {
    p: p.toString(),
    q: q.toString(),
    c1: c1.toString(),
```

```
            c2: c2.toString(),
            F1: F1.toString(),
            F2: F2.toString(),
            flag1: flag1,
            flag2: flag2,
            fullFlag: fullFlag
        };
    } catch (error) {
        console.error('错误:', error.message);
        return { error: error.message };
    }
}
```

```
// 执行主函数
console.log('开始执行 RSA 解密...');
const startTime = Date.now();
const result = main();
const endTime = Date.now();
console.log(`\n执行时间: ${(endTime - startTime) / 1000} 秒`);
console.log('最终结果:');
console.log(JSON.stringify(result, null, 2));
```

1zvm

1、打开网页进入沙箱。

2、把问题输给 trae。

3、把代码复制到网页。

4、调试，把错误结果复制给 ai，再把生成的代码复制到沙箱，直至跳出 flag。

（trae 不保留对话记录，退出一次之后当时的代码也找不到了）

A.故障机器人

1、用 c 语言写出代码雏形。

```
#include<stdio.h>
#include<stdlib.h>
int cmp(const void *a, const void *b) {
	return *(int*)a-*(int*)b;
}
int main(){
	int n,m,i,r;
	scanf("%d",&n);
	m=n;
	int a[100000];
	for(i=0;i<n;i++) {
		scanf("%d",&a[i]);
	}
	long long total_damage=0;
	  qsort(a,n,sizeof(int),cmp);
	 for(i=0;i<n&&m>0;i++)
	  {r=(a[i]+2)/3;
	   total_damage+=m*r-1;
	   m--;
	  }
	printf("%lld\n",total_damage);
	return 0;
}
```

2、把问题输给 trae，生成 cpp 文件。

3、新建.cpp 文件，复制代码。

4、调试，把错误结果复制给 ai，apply，直至通过测试。

最终程序：

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
	ios::sync_with_stdio(false);
	cin.tie(NULL);
```

```
    int n, m, i;
    cin >> n;
    vector<long long> a(n);
    for (i = 0; i < n; ++i) {
        cin >> a[i];
    }
    sort(a.begin(), a.end());

    long long total_damage = 0,r;
    for (i = 0; i < n ; ++i) {
        r = (a[i] + 2) / 3;
        long long c=n-i;
        total_damage += c*r-1;
    }
    cout << total_damage << endl;
    return 0;
}
```

B.QwQwQwQ

1、用 c 语言写出代码雏形。

2、把问题输给 trae，生成 cpp 文件。

3、新建.cpp 文件，复制代码。

初稿：

```
#include <iostream>
#include <string>
#include <vector>
#include <utility>
using namespace std;
long long pow3[31];
void initPow3() {
	pow3[0] = 1;
	for (int i = 1; i <= 30; ++i) {
		pow3[i] = pow3[i-1] * 3;
	}
}

char findChar(const string& s, int k, long long x) {
	for (int i = 0; i < (int)s.size(); ++i) {
		char c = s[i];
		long long len = (c == 'q') ? pow3[k] : 1;
		if (x > len) {
			x -= len;
			continue;
		}
		if (c != 'q') {
			return c;
		} else {
			if (k == 0) {
				return 'q';
			}
			return findChar("qwq", k-1, x);
		}
	}
	return '0';
```

```
}

int main() {
	initPow3();
	ios::sync_with_stdio(false);
	cin.tie(NULL);

	string s;
	cin >> s;
	int m;
	cin >> m;

	vector<pair<int, long long> > queries(m);
	for (int i = 0; i < m; ++i) {
		cin >> queries[i].first >> queries[i].second;
	}

	string result;
	for (int i = 0; i < (int)queries.size(); ++i) {
		int k = queries[i].first;
		long long x = queries[i].second;
		if (k > 30) k = 30;
		char c = findChar(s, k, x);
		result += (c == '0') ? '0' : c;
	}

	cout << result << endl;
	return 0;
}
```

4、调试，把错误结果复制给 ai，apply，直至通过测试。

最终程序：

```
#include <iostream>
#include <string>
#include <vector>
#include <utility>
#include <climits>
using namespace std;
```

```
long long getQlen(int k) {
	long long len = 1;
	for (int i = 0; i < k; ++i) {
		if (len > (LLONG_MAX - 1) / 2) {
			return LLONG_MAX;
		}
		len = 2 * len + 1;
	}
	return len;
}
char findChar(const string& s, int k, long long x) {
	string currentStr = s;
	int currentK = k;
	long long currentX = x;
	while (true) {
		bool found = false;
		for (int i = 0; i < (int)currentStr.size(); ++i) {
			char c = currentStr[i];
			long long len;
			if (c == 'q') {
				len = getQlen(currentK);
			} else {
				len = 1;
			}
			if (currentX <= len) {
				if (c != 'q' || currentK == 0) {
					return c;
				} else {
					currentStr = "qwq";
					currentK -= 1;
					found = true;
					break;
				}
			}
			currentX -= len;
		}
		if (!found) {
```

```
                return '0';
            }
        }
}
int main() {
        ios::sync_with_stdio(false);
        cin.tie(NULL);
        string s;
        cin >> s;
        int m;
        cin >> m;
        vector<pair<int, long long> > queries(m);
        for (int i = 0; i < m; ++i) {
            cin >> queries[i].first >> queries[i].second;
        }
        string result;
        for (int i = 0; i < (int)queries.size(); ++i) {
            int k = queries[i].first;
            long long x = queries[i].second;
            if (x < 1) {
                result += '0';
                continue;
            }
            if (k > 30) k = 30;
            char c = findChar(s, k, x);
            result += (c == '0') ? '0' : c;
        }
        cout << result << endl;
        return 0;
}
```